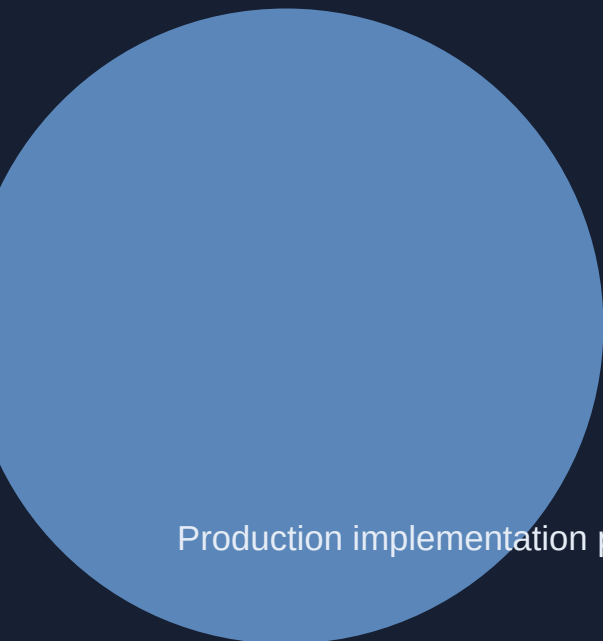




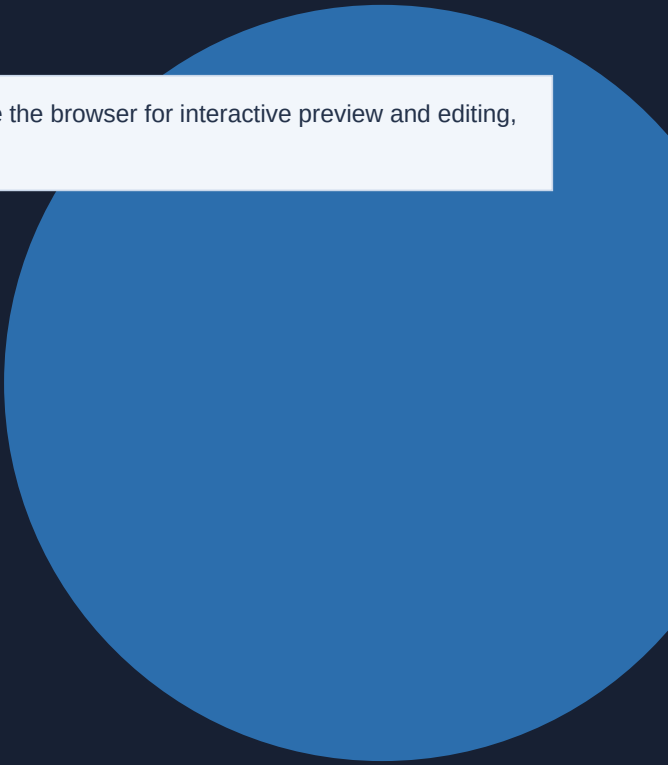
LyricSync

Production implementation plan for an AI-assisted synchronized lyrics video web application

Flask + OpenAI API + Vanilla JavaScript + Bootstrap + FFmpeg/ASS

Deep-research technical blueprint

Prepared: 20 September 2026



Core principle: use timestamped JSON as the single source of truth, use the browser for interactive preview and editing, and use FFmpeg for deterministic final rendering.

1. Executive summary

LyricSync is a web application that accepts an audio track and a background video, transcribes the audio, derives word- or phrase-level timing, presents the lyrics in sync with the video in a browser editor, and renders a final MP4 with the lyrics burned into the video. The initial concept is sound, but several details need to be corrected for the current OpenAI API surface.

The most important correction is model capability. Current OpenAI API documentation states that ``timestamp_granularities`` requires ``verbose_json``, and that this timestamp-granularity feature is supported with ``whisper-1``. The newer GPT-4o transcription models use a JSON response format and do not expose the same timestamp-granularity interface. Therefore, the MVP should use ``whisper-1`` when word timestamps are a hard requirement. The architecture should keep the transcription service abstract so a future timestamp-capable model or second-pass alignment method can be introduced without changing the rest of the product. [1][2]

Engineering conclusion: do not make SRT the primary data model. Keep normalized word-level timestamps in JSON. Generate SRT, WebVTT, or ASS from that JSON only when needed.

The final system should be built as a staged pipeline: upload and validate media, transcribe, normalize, segment words into lyric lines, preview in the browser, allow manual correction, generate ASS subtitles and/or direct FFmpeg filters, render in a background worker, validate the output, and present the completed MP4.

For production, a synchronous Flask endpoint should not be responsible for several-minute renders. Use a job queue such as Celery with Redis or another supported broker/backend. Current Celery documentation lists Redis as a stable broker and backend option, while warning that Redis is best for small messages and should not be treated as a long-term media store. [3]

2. Research findings and design corrections

2.1 OpenAI transcription capabilities

Model / route	Current documented behavior	Implication for LyricSync
whisper-1	Supports <code>`verbose_json`</code> ; supports <code>`timestamp_granularities`</code> with <code>`word`</code> or <code>`segment`</code> .	Use for the MVP when true word-level timestamps are required.
gpt-4o-transcribe	Current docs describe JSON output; the same <code>`timestamp_granularities`</code> interface is not available.	Good candidate for transcript quality experiments, but not the direct word-timestamp source in this design.
gpt-4o-mini-transcribe	Current docs describe JSON output; the same timestamp-granularity interface is not available.	Useful for lower-cost transcript quality experiments or a second pass.
gpt-transcribe	Current docs describe high-accuracy file transcription and keyword hints.	Potential future text-quality route; keep behind an adapter.
gpt-4o-transcribe-diarize	Supports speaker-aware diarized output but does not provide the same word timestamp option.	Relevant only if later versions support multi-speaker dialogue or staged performances.

The official transcription reference also lists supported input containers such as flac, mp3, mp4, mpeg, mpga, m4a, ogg, wav, and webm. The application should still normalize media internally to a predictable format before transcription or rendering. [2]

2.2 Timestamp accuracy is an engineering target, not a guarantee

Word timestamps are model estimates. For ordinary speech they can be very useful, but music introduces sustained vowels, melisma, background instruments, doubled vocals, ad-libs, and ambiguous word boundaries. Consequently, the product should promise an editable synchronization workflow rather than a claim of mathematical perfection.

A practical acceptance target for the MVP is: automatic alignment good enough to make the majority of a song usable without manual work, plus fast manual adjustment for exceptions. Record an internal timing error metric such as median absolute timing error against a manually curated evaluation set.

2.3 Browser preview versus final render

Use HTML5 video plus a synchronized DOM overlay for the editor preview. The `currentTime` property exposes playback time in seconds, and the `timeupdate` event is widely available. However, MDN notes that `timeupdate` frequency varies with system load, roughly 4 Hz to 66 Hz. For high-quality karaoke highlighting, the editor should use a lightweight animation loop based on `requestAnimationFrame` while the video is playing and fall back to `timeupdate` for seeking and state recovery. [4][5]

2.4 Why ASS is preferred over SRT for export

FFmpeg has an `ass` filter that renders ASS subtitles onto video through libass. ASS supports far richer styling than SRT, including font selection, colors, alignment, borders, shadows, and animated override tags. This makes it a good interchange format for karaoke-style rendering. [6][7]

3. Product goals, non-goals, and assumptions

3.1 Primary goals

- Upload an audio track and a background video with clear validation and predictable storage.
- Generate a transcript with word-level timing using a documented timestamp-capable transcription route.
- Convert words into visually coherent lyric lines while preserving word timings.
- Provide an interactive browser preview without rendering a new video for every style change.
- Allow manual timing and text corrections before export.
- Render an H.264/AAC MP4 with configurable lyric style and aspect ratio.
- Track long-running jobs and expose meaningful progress to the frontend.

3.2 Non-goals for MVP

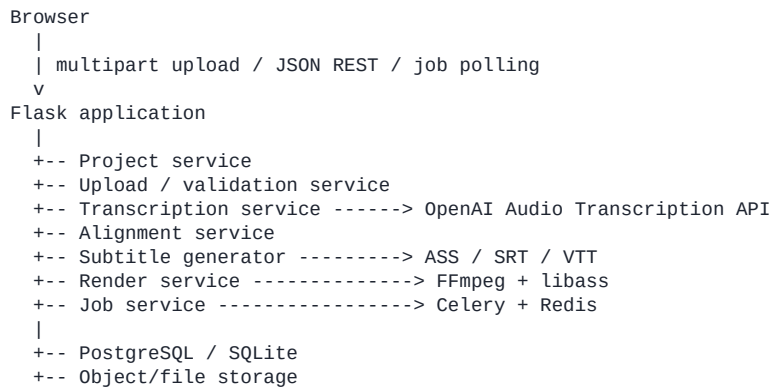
- Real-time microphone transcription.
- Automatic beat tracking as a dependency for every export.
- Advanced motion-graphics compositing comparable to professional desktop editors.
- Multi-user collaboration and enterprise organization features.
- Automatic copyright detection or rights management.

3.3 Assumptions

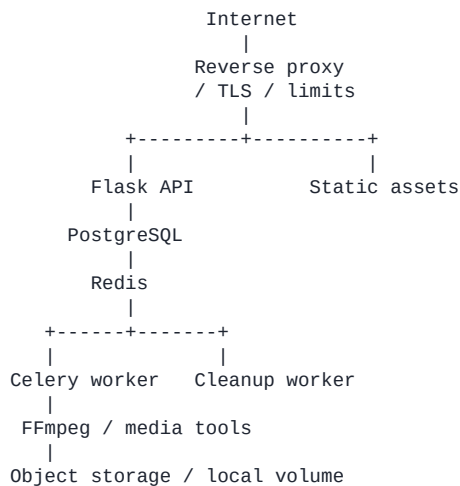
- The user owns or is licensed to use the uploaded audio and video.
- The server has FFmpeg with the required filters and codecs installed.
- The application keeps the original media and all derived artifacts associated with a project.
- The MVP is initially single-tenant or uses a simple authenticated user model.

4. System architecture

4.1 Logical architecture



4.2 Production deployment shape



4.3 Component responsibilities

Component	Responsibility	Should not own
Frontend	Upload UX, playback, editor state, preview rendering, job polling.	Secrets, FFmpeg execution, filesystem access.
Flask API	Authentication, validation, project lifecycle, orchestration, persistence.	Long-running encoding in request threads.
Transcription service	OpenAI request, retries, response normalization, model adapter.	Video styling or FFmpeg commands.
Alignment service	Word cleanup, line grouping, timing normalization, correction rules.	Rendering or user authentication.
Subtitle service	Generate ASS/SRT/VTT from canonical JSON.	Calling OpenAI directly.
Render service	Construct safe FFmpeg argument arrays and encode output.	Arbitrary shell string construction.
Job system	Execute long-running work asynchronously, report status.	Permanent media storage.
Database	Project metadata, job states, settings, audit information.	Large media blobs.

5. Canonical data model

The most important data contract is the normalized project JSON. Every downstream representation should be derived from it. This prevents timing drift caused by maintaining separate, loosely synchronized SRT files, frontend arrays, and database text copies.

5.1 Canonical project document

```
{
  "project_id": "01J...",
  "media": {
    "audio_path": "...",
    "video_path": "...",
    "audio_duration": 245.20,
    "video_duration": 180.00,
    "fps": 30,
    "width": 1920,
    "height": 1080
  },
  "transcription": {
    "provider": "openai",
    "model": "whisper-1",
    "language": "en",
    "raw_text": "...",
    "version": 1
  },
  "lyrics": [
    {
      "id": 1,
      "text": "I love you tonight",
      "start": 12.42,
      "end": 15.27,
      "words": [
        {"text": "I", "start": 12.42, "end": 12.61},
        {"text": "love", "start": 12.62, "end": 13.21},
        {"text": "you", "start": 13.22, "end": 13.64},
        {"text": "tonight", "start": 13.65, "end": 15.27}
      ]
    }
  ],
  "style": {
    "mode": "karaoke",
    "font": "Arial",
    "font_size": 48,
    "position": "bottom",
    "primary_color": "#FFFFFF",
    "highlight_color": "#FFE66D",
    "outline": 2,
    "shadow": 1
  },
  "render": {
    "aspect_ratio": "16:9",
    "resolution": "1920x1080",
    "audio_policy": "replace",
    "video_policy": "loop"
  }
}
```

5.2 Why words and lines both matter

A word-level list supports karaoke effects and precise highlighting. A line object supports readable layout and editor operations. The line start should normally be the first contained word start, while line end is the final contained word end. When the editor changes timing, update the line and its affected words consistently.

5.3 Versioning

Every transcription and alignment result should carry a version. Manual edits create a new alignment revision rather than overwriting the original AI result. This enables undo, auditability, reproducibility, and safer experimentation with improved algorithms.

6. Database design

6.1 Core tables

Table	Key fields	Purpose
users	id, email, password_hash, created_at	Authentication and ownership.
projects	id, user_id, name, status, created_at, updated_at	Project lifecycle.
media_assets	id, project_id, kind, storage_key, mime_type, size_bytes, duration, checksum	Audio/video and derived assets.
transcriptions	id, project_id, model, language, raw_text, json_key, version	Raw and normalized transcription metadata.
lyric_lines	id, project_id, revision, line_index, text, start, end	Editable line-level lyrics.
lyric_words	id, line_id, word_index, text, start, end	Word timing records.
style_presets	id, user_id, name, config_json	Reusable typography styles.
render_jobs	id, project_id, status, progress, celery_task_id, output_asset_id, error_code	Asynchronous export state.
audit_events	id, user_id, project_id, event_type, payload_json, created_at	Operational trace.

6.2 PostgreSQL recommendation

SQLite is ideal during local development. For production, PostgreSQL is the better default because projects, render jobs, revisions, and concurrent users benefit from transactional consistency and indexing. SQLAlchemy 2.x provides the ORM layer and is currently in the 2.0 series; the official documentation recommends the modern declarative mapping style. [8]

7. Recommended repository structure

```
lyricsync/  
  ■■■ app.py  
  ■■■ config.py  
  ■■■ requirements.txt  
  ■■■ .env.example  
  ■■■ .gitignore  
  ■■■ docker-compose.yml  
  ■■■ Dockerfile  
  ■■■ migrations/  
  ■■■ app/  
  ■   ■■■ __init__.py  
  ■   ■■■ extensions.py  
  ■   ■■■ models/  
  ■     ■■■ user.py  
  ■     ■■■ project.py  
  ■     ■■■ media_asset.py  
  ■     ■■■ transcription.py  
  ■     ■■■ render_job.py  
  ■   ■■■ routes/  
  ■     ■■■ auth.py  
  ■     ■■■ projects.py  
  ■     ■■■ uploads.py  
  ■     ■■■ lyrics.py  
  ■     ■■■ renders.py  
  ■   ■■■ services/  
  ■     ■■■ openai_transcription.py  
  ■     ■■■ alignment.py  
  ■     ■■■ subtitles.py  
  ■     ■■■ media_probe.py  
  ■     ■■■ ffmpeg.py  
  ■     ■■■ cleanup.py  
  ■   ■■■ tasks/  
  ■     ■■■ transcription_tasks.py  
  ■     ■■■ render_tasks.py  
  ■   ■■■ utils/  
  ■     ■■■ files.py  
  ■     ■■■ validation.py  
  ■     ■■■ ids.py  
  ■■■ templates/  
  ■     ■■■ dashboard.html  
  ■     ■■■ upload.html  
  ■     ■■■ editor.html  
  ■■■ static/  
  ■     ■■■ css/  
  ■     ■■■ js/  
  ■■■ tests/  
  ■     ■■■ test_uploads.py  
  ■     ■■■ test_alignment.py  
  ■     ■■■ test_api.py  
  ■     ■■■ test_rendering.py
```

Keep business rules in services rather than route functions. Routes should parse requests, enforce access control, invoke services, and serialize responses. This makes the application testable and keeps the API layer thin.

8. Media upload and validation pipeline

8.1 Upload flow

```
Browser  
-> POST /api/projects  
-> create project record  
-> validate audio  
-> validate video  
-> store with generated IDs  
-> probe duration / codec / dimensions  
-> return project + upload status
```

8.2 Validation rules

- Allow-list extensions and also validate detected media type. Do not trust the browser Content-Type header.
- Generate the stored filename server-side. Never use the raw user filename as a storage path.
- Enforce application-level size limits before expensive processing.

- Probe the media with FFprobe and reject malformed or unsupported files before transcription/rendering.
- Store media outside the application code directory and preferably outside the web root.
- Use a per-project directory or object-storage prefix to isolate artifacts.

Flask documentation explicitly recommends `secure_filename()` for uploaded filenames and supports `MAX_CONTENT_LENGTH` for request-size limits. OWASP further recommends allow-listing extensions, validating file type, generating server-side names, applying size limits, and storing uploads outside the web root or on a separate storage service. [9][10]

8.3 Suggested application limits

Asset	Development limit	Production starting point	Notes
Audio	50 MB	100 MB	Choose a lower limit than infrastructure can tolerate.
Video	200 MB	1 GB	Prefer direct-to-object-storage upload at larger scale.
Project duration	10 min	30-60 min	Start lower until job/queue behavior is proven.
Concurrent renders/user	1	1-2	Prevents one account from monopolizing workers.

These limits are product policy suggestions, not upstream OpenAI limits. Reconfirm provider and hosting limits before launch.

9. OpenAI transcription service

9.1 Recommended MVP implementation

Use a dedicated adapter so the rest of the system does not care whether transcription comes from `whisper-1` or a future service.

```
from openai import OpenAI

class OpenAITranscriber:
    def __init__(self, api_key: str):
        self.client = OpenAI(api_key=api_key)

    def transcribe_word_timestamps(self, audio_path: str):
        with open(audio_path, "rb") as audio_file:
            result = self.client.audio.transcriptions.create(
                model="whisper-1",
                file=audio_file,
                response_format="verbose_json",
                timestamp_granularities=["word"],
            )

        return {
            "text": result.text,
            "language": getattr(result, "language", None),
            "duration": getattr(result, "duration", None),
            "words": [
                {
                    "text": w.word,
                    "start": float(w.start),
                    "end": float(w.end),
                }
                for w in (result.words or [])
            ],
        }
```

The current OpenAI documentation gives the same Python pattern: `whisper-1`, `verbose_json`, and `timestamp_granularities=["word"]`. It also notes that word timestamps add latency. [2][11]

9.2 Optional second-pass quality architecture

A future version can decouple text quality from timing. For example: transcribe once with `whisper-1` to get word boundaries, optionally transcribe again with a newer model for a cleaner transcript, then use sequence alignment between the two text streams to transfer or reconcile wording while preserving the original timing track. This should only be introduced after the baseline evaluation set shows a measurable quality gain.

9.3 Retry strategy

- Retry transient provider/network failures with exponential backoff and jitter.
- Do not retry malformed media or deterministic validation failures.
- Persist provider error category, attempt count, and last failure time.
- Make the job idempotent so a retry does not create duplicate projects or conflicting outputs.

10. Synchronization and lyric alignment engine

10.1 Stage A - normalize words

Input: provider word objects

For each word:

1. strip leading/trailing whitespace
2. preserve original token text for audit
3. clamp negative values to zero
4. ensure end >= start
5. discard zero-information artifacts
6. attach stable word index

10.2 Stage B - line segmentation

Line segmentation should be deterministic. Do not ask a language model to decide all line boundaries in production unless evaluation proves it materially better. The first implementation can use word count, punctuation, pause duration, and maximum line width heuristics.

Heuristic	Example threshold	Purpose
Max words	6-9 words	Keep lyric lines readable.
Long pause	0.60-0.90 s	Start a new line around musical gaps.
Sentence punctuation	., !, ?	Natural segmentation when present.
Max visible characters	28-42 chars	Avoid excessively wide lines.
Very short line	< 2 words	Merge with adjacent line when visually appropriate.

10.3 Stage C - word activation

For each frame of the preview, identify the active line and active word. Maintain a current index and advance while the next word start is <= currentTime. This avoids repeatedly scanning the entire lyric list.

```
let wordIndex = 0;

function update(time) {
  while (wordIndex + 1 < words.length &&
    words[wordIndex + 1].start <= time) {
    wordIndex++;
  }

  const w = words[wordIndex];
  const active = w && time >= w.start && time <= w.end;
  renderWordState(wordIndex, active);
}
```

10.4 Seek handling

When the user drags the playback bar, the sequential pointer may become invalid. On seek, binary-search the word array by start time and reset the pointer. This keeps performance near $O(\log n)$ for seeks and $O(1)$ amortized for continuous playback.

10.5 Manual timing correction

- Editable start and end times per line.
- Optional per-word nudge controls for difficult sections.
- Snap only within a small, configurable tolerance, never as a hidden behavior.
- Save a new alignment revision when the user publishes an export.

11. Frontend architecture and UX

11.1 Application screens

Screen	Primary purpose	Core controls
Dashboard	Project management.	New project, search, status, open/delete.
Upload	Collect media safely.	Audio drop zone, video drop zone, validation feedback.
Editor	Preview and synchronization.	Video player, lyrics overlay, timeline, style panel.
Render	Track export.	Progress, cancel, retry, completed download.

11.2 Editor layout

Project name		[Save] [Export video]
VIDEO LYRIC OVERLAY	Typography	
	Font	
	Size	
	Color	
	Highlight	
	Position	
	Style / Animation	
	Aspect ratio	
00:00 waveform / timeline / lyric blocks		04:20
[play] [pause] [seek]		Current lyric / word inspector

11.3 Bootstrap usage

Bootstrap 5.3 is appropriate for the shell UI: forms, progress, responsive layout, dialogs, and utility classes. Bootstrap 5.3 also uses regular JavaScript rather than requiring jQuery, which fits the vanilla frontend requirement. [12][13]

11.4 Preview rendering

The preview should never encode the video. Put the video in a positioned container and layer an accessible lyric overlay above it. The lyric overlay can be implemented as HTML elements so active-word styling and line transitions are inexpensive during editing.

12. REST API design

12.1 Project endpoints

Method	Endpoint	Purpose
POST	/api/projects	Create project and register uploads.
GET	/api/projects	List projects for current user.
GET	/api/projects/{id}	Return project metadata and current revision.
DELETE	/api/projects/{id}	Delete project and queue storage cleanup.
POST	/api/projects/{id}/transcribe	Queue transcription job.
GET	/api/projects/{id}/lyrics	Return canonical lyric JSON.
PUT	/api/projects/{id}/lyrics	Save edited lyric revision.
PUT	/api/projects/{id}/style	Update style configuration.
POST	/api/projects/{id}/render	Queue final render.
GET	/api/jobs/{id}	Return status and progress.
GET	/api/projects/{id}/download	Authorize and serve final artifact.

12.2 Example response

```
{
  "success": true,
  "project": {
    "id": "p_8f31...",
    "status": "ready",
    "audio": {"duration": 245.2},
    "video": {"duration": 180.0},
    "lyrics_revision": 3
  }
}
```

12.3 Error contract

```
{
  "success": false,
  "error": {
    "code": "MEDIA_UNSUPPORTED",
    "message": "The uploaded video codec is not supported.",
    "retryable": false
  }
}
```

Use stable machine-readable error codes. Human-readable text can change without breaking the frontend.

13. FFmpeg rendering pipeline

13.1 Render stages

1. Probe source media
2. Decide output duration policy
3. Normalize/scale background video
4. Select or replace audio stream
5. Generate ASS subtitles from canonical lyric JSON
6. Apply ASS via libass / FFmpeg
7. Encode H.264 video + AAC audio
8. Faststart MP4 for web delivery
9. Validate output duration, streams, and decodeability
10. Store final artifact and mark job complete

13.2 Why direct FFmpeg

MoviePy is convenient for quick Python experiments, but direct FFmpeg commands are preferable for the production renderer because the application needs explicit control over codecs, mappings, filters, duration behavior, and subtitle rendering. FFmpeg documentation exposes stream mapping and options such as `-shortest`; its filter documentation includes looping and subtitle filters. [6][14]

13.3 Safe command construction

```
subprocess.run([
    "ffmpeg",
    "-y",
    "-i", video_path,
    "-i", audio_path,
    "-vf", f"ass={ass_path_escaped}",
    "-map", "0:v:0",
    "-map", "1:a:0",
    "-c:v", "libx264",
    "-preset", "medium",
    "-crf", "20",
    "-c:a", "aac",
    "-b:a", "192k",
    "-movflags", "+faststart",
    output_path,
], check=True)
```

In a real implementation, escape the subtitle path for the filter syntax or use an approach that avoids fragile inline filter strings. Never construct a whole shell command using string interpolation and then invoke it with `shell=True`.

13.4 Duration policy

Case	Recommended behavior
Video shorter than audio	Loop the video or freeze the last frame, selectable by the user.
Video longer than audio	Trim video to audio length by default.
Video has unwanted audio	Use uploaded track as final audio.
User wants mixed audio	Provide explicit volume controls and ducking rules.
Different aspect ratio	Scale and crop to a declared output canvas.

14. ASS subtitle generation and karaoke rendering

14.1 ASS style mapping

UI setting	ASS representation
Font family	FontName
Font size	FontSize
Primary text	PrimaryColour
Inactive / secondary text	SecondaryColour
Outline	Outline
Shadow	Shadow
Top/center/bottom	Alignment + margins
Word highlight	Karaoke override timing or multiple timed dialogue events

libass is specifically designed to render ASS/SSA subtitles and documents support for styling and override tags. FFmpeg exposes an `ass` filter to render ASS subtitles using libass. [6][7]

14.2 Two export strategies

- **Strategy A - timed dialogue events:** generate frequent subtitle events so the currently active word or phrase is styled differently. This is simpler to reason about but may generate many events.

- **Strategy B - native ASS karaoke effects:** use ASS override tags for progressive highlight effects. This is more compact and can yield richer karaoke behavior, but requires careful timing conversion and more renderer-specific testing.

14.3 MVP recommendation

Start with timed dialogue events for classic and simple word-highlight modes. Add native ASS karaoke tags only after the basic rendering path is stable. This reduces the number of simultaneous variables during early debugging.

15. Asynchronous job processing

15.1 Why a job queue is required

Transcription and especially video encoding can outlive an HTTP request. Queue these operations. The API creates the job and immediately returns a job identifier. A worker updates progress and the frontend polls or subscribes to status.

15.2 Celery + Redis topology

```
Flask -> Redis broker -> Celery worker -> task
|
+--> transcription task
+--> alignment task
+--> render task
+--> cleanup task
```

Celery 5.6 documentation identifies Redis as a stable broker and describes Redis as an effective backend for task state, while noting that Redis is not a media archive. Store media elsewhere. [3]

15.3 Job states

State	Meaning
queued	Waiting for a worker.
running	Worker has started.
transcribing	OpenAI request in progress.
aligning	Canonical lyrics are being built.
rendering	FFmpeg encoding.
validating	Output file checks running.
completed	Artifact is ready.
failed	Terminal error with code and log reference.
cancelled	User or system cancelled the job.

15.4 Progress model

For an MVP, progress can be stage-based rather than claiming exact percentage from FFmpeg unless you parse FFmpeg progress output. A practical scale is: upload 0-15%, transcription 15-40%, alignment 40-50%, render 50-95%, validation 95-100%.

16. Browser synchronization implementation

16.1 Basic editor loop

```
const video = document.querySelector('#video');
const lyricLines = project.lyrics;
let currentWordIndex = 0;

function seekToWordIndex(time) {
  // Binary search for greatest word.start <= time.
  let lo = 0, hi = allWords.length - 1, ans = 0;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (allWords[mid].start <= time) {
      ans = mid;
      lo = mid + 1;
    } else {
      hi = mid - 1;
    }
  }
  currentWordIndex = ans;
}

function render() {
  const t = video.currentTime;
  const word = allWords[currentWordIndex];
  if (word && t >= word.start && t <= word.end) {
    paintActiveWord(currentWordIndex);
  } else if (word && t > word.end) {
    while (currentWordIndex + 1 < allWords.length &&
      allWords[currentWordIndex + 1].start <= t) {
      currentWordIndex++;
    }
    paintActiveWord(currentWordIndex);
  }
  if (!video.paused && !video.ended) {
    requestAnimationFrame(render);
  }
}

video.addEventListener('play', () => requestAnimationFrame(render));
video.addEventListener('seeking', () => seekToWordIndex(video.currentTime));
video.addEventListener('timeupdate', () => {
  // recovery hook; rAF handles high-frequency visual updates
});
```

This design uses `currentTime` as the authoritative playback clock. `timeupdate` remains useful for recovery and state changes, but the visual loop can be driven by `requestAnimationFrame` while playback is active. [4][5]

16.2 Accessibility

- Provide an accessible transcript panel separate from the decorative overlay.
- Do not communicate synchronization meaning only through color; also use weight, underline, or a non-visual state.
- Respect `prefers-reduced-motion` for animated lyric transitions.
- Make all editor controls keyboard reachable.

17. Timeline and manual editing

17.1 Minimum timeline features

- Click a lyric line to seek the video to its start.
- Drag line start and end handles.
- Edit lyric text inline.
- Split a line into two lines.
- Merge adjacent lines.
- Nudge selected line earlier or later by configurable milliseconds.

- Undo and redo revisions.

17.2 Data integrity rule

Never let the UI modify one timestamp independently without updating the canonical representation. The editor should send a complete revision or a typed patch, and the server should validate that every word remains inside its parent line and that times are monotonically increasing.

17.3 Proposed editor operations

Operation	Validation
Move line start	$0 \leq \text{start} < \text{end} \leq \text{project duration}$.
Move line end	$\text{start} < \text{end}$ and no accidental overlap unless overlaps are allowed.
Split line	Each new line contains at least one word.
Merge lines	Words remain ordered by start time.
Edit text	Preserve word timing unless the editor explicitly enters word-level correction mode.
Delete words	Require explicit confirmation because it changes transcript semantics.

18. Security and abuse prevention

18.1 Upload security

- Allow-list file extensions and inspect actual media type.
- Use generated filenames and opaque project identifiers.
- Enforce size and duration limits.
- Store media outside the web root and serve through authorized download endpoints or signed URLs.
- Run media probing in a constrained worker environment.
- Consider antivirus or media-file scanning at scale.

OWASP recommends treating file uploads as untrusted input, generating filenames server-side, applying size limits, validating actual file types, and keeping uploaded content away from direct execution paths. [10]

18.2 API security

- Keep `OPENAI\_API\_KEY` only on the server.
- Authenticate every project mutation and download.
- Apply rate limits to transcription and rendering endpoints.
- Do not expose provider error payloads verbatim to the browser if they reveal secrets or internal paths.
- Use CSRF protection if cookie-based authentication is used.
- Record audit events for uploads, exports, and deletes.

18.3 FFmpeg execution security

Never concatenate user-controlled paths into a shell command. Use argument arrays with `subprocess.run`. Treat subtitle content as data and sanitize style parameters before generating ASS. Limit worker filesystem permissions so a compromised media process cannot access secrets or unrelated files.

19. Storage, cleanup, and retention

19.1 Artifact lifecycle

```
Original audio
Original video
|
+-- normalized/probed metadata
+-- transcription.json
+-- aligned_lyrics_vN.json
+-- preview state (database)
+-- ASS subtitle
+-- rendered MP4
+-- thumbnail / waveform (optional)
```

19.2 Retention policy

Large media should not remain indefinitely. Define policy before production. Example: drafts retained 30 days after last activity, completed exports retained 7-30 days unless the user explicitly saves them, failed temporary artifacts removed after 24 hours. Keep database metadata and audit records longer if operationally required.

19.3 Object storage at scale

For a small deployment, local volumes can work. For multiple workers or multiple application instances, move media to S3-compatible object storage or another managed object store. Workers should download only the assets required for their job and write results back to object storage.

20. Observability and operational reliability

20.1 Structured logs

```
{
  "event": "render_completed",
  "project_id": "p_8f31",
  "job_id": "j_77ab",
  "duration_sec": 245.2,
  "render_seconds": 68.4,
  "output_bytes": 48392011,
  "ffmpeg_exit_code": 0
}
```

20.2 Metrics

Metric	Why it matters
transcription_latency_seconds	Provider and queue performance.
transcription_failure_rate	Provider/media reliability.
alignment_revision_rate	How often users need manual correction.
render_seconds_per_media_minute	Encoder efficiency.
render_failure_rate	Production stability.
job_queue_depth	Capacity pressure.
storage_bytes_per_user	Retention and cost control.
median_timing_error_ms	Core product quality metric.

20.3 Health checks

- Flask health endpoint: database connectivity and basic application status.
- Worker health: Celery worker heartbeat and Redis reachability.
- Media tool check: FFmpeg/FFprobe version and required filter availability.

21. Testing strategy and acceptance criteria

21.1 Unit tests

- Word normalization tests.
- Line segmentation boundary tests.
- Binary-search seek logic tests.
- ASS time conversion tests.
- Aspect-ratio math tests.
- Duration policy tests.
- Filename/path validation tests.

21.2 Integration tests

- Upload -> project creation -> media probe.
- Project -> transcription -> canonical JSON.
- Canonical JSON -> ASS generation.
- Project -> render job -> output validation.

21.3 End-to-end golden test set

Build a small manually annotated benchmark containing clean speech, singing, fast lyrics, repeated chorus lines, instrumental gaps, multi-language content, and noisy recordings. Store ground-truth line and word timings for a subset. Every change to transcription or alignment logic should run against this benchmark.

21.4 Acceptance criteria for MVP

Area	Acceptance condition
Upload	Valid audio/video completes upload; invalid formats return actionable errors.
Transcription	Timestamped words are persisted for supported test files.
Preview	Active lyrics track the video clock during play and seek.
Editing	User can correct at least line timing and text.
Render	Export produces a playable MP4 with synchronized audio and burned-in lyrics.
Reliability	Failed provider or render job is visible and retryable where safe.
Security	No arbitrary file path or shell execution from user input.

22. Deployment plan

22.1 Local development

```
Python 3.x
Flask
SQLite
Redis
FFmpeg / FFprobe
OpenAI API key in .env
Browser: current Chrome / Firefox / Edge
```

22.2 Docker Compose development stack

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
      - redis
      - db

  worker:
    build: .
    command: celery -A app.celery_app worker --loglevel=INFO
    env_file: .env
    depends_on:
      - redis
      - db

  redis:
    image: redis:7

  db:
    image: postgres:18
    environment:
      POSTGRES_DB: lyricsync
      POSTGRES_USER: lyricsync
      POSTGRES_PASSWORD: change-me
```

Docker documentation provides current Compose guidance for Flask and Redis applications and describes production adjustments such as environment configuration and restart policies. [15][16]

22.3 Production topology

- Reverse proxy / TLS termination.
- Gunicorn or another production WSGI server for Flask.
- Separate Celery worker processes from web processes.
- Managed PostgreSQL.
- Managed Redis.
- Object storage for media and exports.
- Centralized logs and metrics.
- Automated database migrations and deployment health checks.

23. Configuration and secrets

23.1 Recommended `.env`

```
FLASK_ENV=production
SECRET_KEY=replace-this
DATABASE_URL=postgresql+psycopg://user:password@db/lyricsync
REDIS_URL=redis://:password@redis:6379/0
OPENAI_API_KEY=replace-this
OPENAI_TRANSCRIPTION_MODEL=whisper-1
UPLOAD_MAX_AUDIO_MB=100
UPLOAD_MAX_VIDEO_MB=500
MAX_PROJECT_DURATION_SECONDS=3600
MEDIA_ROOT=/data/media
OUTPUT_ROOT=/data/outputs
```

Secrets must not be committed to Git. Provide `.env.example` without real credentials. Use the hosting platform's secret manager in production where available.

23.2 Model configuration rule

Do not hardcode `whisper-1` throughout the application. Keep the model configurable in the transcription adapter, but preserve capability checks so a developer cannot accidentally switch to a model that does not provide the timestamp format required by the alignment engine.

24. Cost and capacity model

24.1 OpenAI cost considerations

At the time of this research, the official OpenAI model pages list `whisper-1` at \$0.006 per minute. They also list GPT Transcribe at \$0.0045 per minute and GPT-4o Mini Transcribe / GPT-4o Transcribe with token-based audio pricing. Because the timestamp requirement currently points to `whisper-1`, the MVP cost model should use the Whisper per-minute rate and treat other model costs as optional future experiments. Recheck the official pricing pages before launch because prices and model capabilities can change. [17][18][19]

24.2 Example transcription cost model

Monthly audio minutes	Whisper rate used for planning	Approx. transcription spend
1,000	\$0.006/min	\$6
10,000	\$0.006/min	\$60
100,000	\$0.006/min	\$600

These are planning examples derived from the listed rate, not a forecast of total product cost. Encoding, object storage, bandwidth, database, Redis, observability, and compute add to the total cost.

24.3 Rendering capacity

Measure rendering as an encoding ratio, for example $\text{render_seconds} / \text{media_minutes}$. Use this observed value to estimate worker concurrency. A worker that renders one five-minute video in 60 seconds has a 0.2x real-time encoding ratio; ten such workers would theoretically process about 50 minutes of source media per minute before overhead, storage latency, and queue effects are considered.

25. Implementation roadmap

25.1 Phase 0 - environment and baseline (1-2 days)

- Create repository and Flask app factory.
- Configure `.env`, OpenAI client, SQLAlchemy, migrations.
- Install FFmpeg/FFprobe and verify ASS filter availability.
- Create one smoke test that transcribes a short local audio fixture.

25.2 Phase 1 - upload MVP (2-3 days)

- Create project model and upload endpoint.
- Validate extension, MIME, size, and media metadata.
- Store generated files and display upload state.

25.3 Phase 2 - transcription and canonical JSON (2-4 days)

- Implement `whisper-1` timestamped transcription adapter.
- Normalize words.
- Persist raw response and canonical JSON.

- Add retry and error mapping.

25.4 Phase 3 - alignment engine (3-5 days)

- Implement deterministic line segmentation.
- Add pause/punctuation/max-width heuristics.
- Create revision schema.
- Write benchmark tests.

25.5 Phase 4 - browser preview (3-5 days)

- HTML5 video player.
- Lyric overlay.
- Sequential playback pointer + binary-search seek.
- Classic and karaoke display modes.

25.6 Phase 5 - editor (4-6 days)

- Timeline UI.
- Line split/merge.
- Timing nudges.
- Text corrections.
- Undo/redo revisions.

25.7 Phase 6 - render pipeline (3-6 days)

- ASS generator.
- FFmpeg render service.
- Duration and aspect ratio policies.
- Output validation.

25.8 Phase 7 - background jobs (2-4 days)

- Celery + Redis.
- Job state persistence.
- Retries, cancellation, cleanup.
- Frontend progress polling.

25.9 Phase 8 - production hardening (5-8 days)

- Auth and access control.
- Object storage.
- Monitoring.
- Retention cleanup.
- Load and security testing.
- Deployment automation.

26. Prioritized feature backlog

Priority	Feature	Reason
P0	Audio/video upload	Core workflow.
P0	Timestamped transcription	Core synchronization dependency.
P0	Canonical lyrics JSON	Single source of truth.
P0	Browser preview	Fast feedback without rendering.
P0	FFmpeg export	Core deliverable.
P1	Manual timing editor	Handles imperfect singing timestamps.
P1	ASS styling	Professional output quality.
P1	Celery job queue	Production reliability.
P1	Multiple aspect ratios	Social/video use cases.
P2	Waveform timeline	Better editing ergonomics.
P2	Beat-synchronized effects	Advanced visual polish.
P2	Style presets	Faster repeated creation.
P3	Accounts, teams, billing	Commercial scale.
P3	Multi-speaker modes	Specialized use cases.

27. Risk register and mitigations

Risk	Impact	Mitigation
Word timestamps drift on singing	High	Manual timing editor; benchmark on real songs; preserve source timing revision.
Provider capability changes	High	Adapter pattern + startup capability checks + pinned tested models.
Large media overwhelms Flask	High	Size limits; object storage; direct upload later; background jobs.
FFmpeg filter/path edge cases	High	Containerized renderer; subprocess arg arrays; golden render tests.
Long render blocks worker	Medium	Separate render workers and resource limits.
Storage growth	Medium	Retention policy; cleanup jobs; object storage lifecycle rules.
Users upload malicious media	High	Validation, isolation, non-executable storage, scanning, least privilege.
Frontend timing stutters	Medium	Use indexed word pointer and requestAnimationFrame; minimize DOM work.
Lyrics line-breaking looks poor	Medium	Deterministic heuristics + manual split/merge.

28. Implementation checklists

28.1 Before MVP demo

- ``.env`` and secrets configured.
- FFmpeg and FFprobe detected.
- One test audio and one test video included.
- Upload validation working.
- Word timestamps persisted.
- Browser preview synchronized.
- One-button MP4 export working.

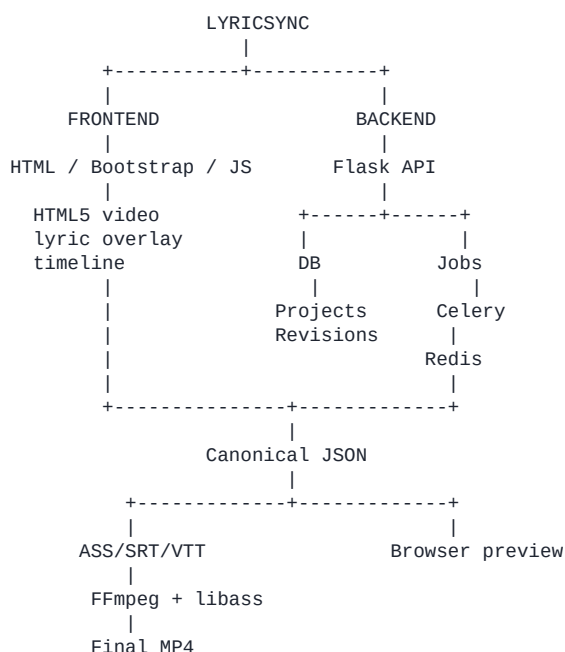
28.2 Before production

- Authentication and authorization.
- PostgreSQL migrations.
- Celery/Redis worker deployment.
- Object storage.
- Rate limits and quotas.
- Centralized logs/metrics.
- Backups and retention.
- Load tests with large files.
- Security review of uploads and FFmpeg execution.
- Documented incident and retry procedures.

28.3 Quality gate for synchronization

Do not accept a new transcription or alignment implementation merely because a demonstration looks good. Compare it against the benchmark. Track at least: word boundary timing error, line boundary error, percentage of words requiring manual adjustment, and render success rate.

29. Final recommended architecture



This architecture preserves a clean separation: OpenAI handles transcription, deterministic Python handles normalization and alignment, the browser handles interactive preview, and FFmpeg handles final media composition. This keeps the system explainable, testable, and extensible.

Recommended implementation order: Upload -> transcription -> canonical word JSON -> line alignment -> browser preview -> manual editor -> ASS generator -> FFmpeg renderer -> Celery/Redis -> production storage and observability.

The single most important product decision is to make the synchronization result editable. Automatic word timestamps should get the user close to the result quickly, while the timeline editor provides the final authority over timing.

30. Research references

- [1] OpenAI, Create transcription API reference. Current documentation covering models, response formats, and timestamp granularity. <https://developers.openai.com/api/reference/cli/resources/audio/subresources/transcriptions/methods/create>
- [2] OpenAI, Audio transcription API reference and speech-to-text documentation. Current model/parameter details and word-timestamp example for whisper-1. <https://developers.openai.com/api/docs/guides/speech-to-text>
- [3] Celery 5.6.3 documentation, Backends and Brokers / Redis. <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/> and <https://docs.celeryq.dev/en/latest/getting-started/backends-and-brokers/redis.html>
- [4] MDN, HTMLMediaElement currentTime. <https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement/currentTime>
- [5] MDN, HTMLMediaElement timeupdate event. https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement/timeupdate_event
- [6] FFmpeg documentation, filters and ASS subtitle rendering. <https://ffmpeg.org/ffmpeg-filters.html>
- [7] libass project documentation and source references for ASS/SSA rendering. <https://github.com/libass/libass>
- [8] SQLAlchemy 2.0 documentation. <https://docs.sqlalchemy.org/en/20/orm/>
- [9] Flask documentation, Uploading Files. <https://flask.palletsprojects.com/en/stable/patterns/fileuploads/>
- [10] OWASP File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- [11] OpenAI Python SDK source/examples for audio transcription. <https://github.com/openai/openai-python>
- [12] Bootstrap 5.3 documentation, Forms. <https://getbootstrap.com/docs/5.3/forms/overview/>
- [13] Bootstrap 5.3 migration documentation, including JavaScript and utility changes. <https://getbootstrap.com/docs/5.3/migration/>
- [14] FFmpeg documentation, stream mapping and output controls. <https://ffmpeg.org/ffmpeg.html>
- [15] Docker, Compose Quickstart for Flask + Redis applications. <https://docs.docker.com/compose/gettingstarted/>
- [16] Docker, Compose production guidance. <https://docs.docker.com/compose/how-tos/production/>
- [17] OpenAI, Whisper model page and current listed transcription price. <https://developers.openai.com/api/docs/models/whisper-1>
- [18] OpenAI, GPT Transcribe model page. <https://developers.openai.com/api/docs/models/gpt-transcribe>
- [19] OpenAI, GPT-4o Transcribe and GPT-4o Mini Transcribe model pages. <https://developers.openai.com/api/docs/models/gpt-4o-transcribe> and <https://developers.openai.com/api/docs/models/gpt-4o-mini-transcribe>

Research date: 20 September 2026. Model names, endpoint capabilities, pricing, and third-party library versions can change. Revalidate these items before production deployment.